Mock Midterm 2, Compilers: Solutions

Ten MCQs with a trap in each, then every technique once (80 points)

Compiler Construction Module

Every scan, automaton, FIRST/FOLLOW set, table and stack trace was recomputed by a script from the data as written.

Summary: How to use it

Do the questions paper first, then compare here. Each MCQ solution names the trap the question was built on, and why the other options fail. Each Part B solution is worked in the order you should write it on the exam.

Part A: Multiple choice with justification (20 pts)

Problem: A1 (2 pts): which phase complains?

The source line is `x = 3y + 1;` in a C-like language whose scanner has the usual rules for identifiers, integers, operators and whitespace. Which phase reports the first problem?
(A) The scanner: `3y` is not a valid number. (B) The parser: the token sequence is wrong. (C) The semantic analyser: `y` is undeclared. (D) No phase: the line is legal.

Solution

(B). The scanner never fails on `3y`: by longest match it returns NUM(3) and then ID(y), both legal tokens; a scanner only rejects a character that starts no rule. The parser then sees NUM ID with no operator between them, which no production derives: a syntax error. (C) would come later and only if the tokens formed a legal statement. Trap: "looks like a bad identifier" is not a lexical error.

Problem: A2 (2 pts): longest match versus rule order

A scanner has, in this order, rule 1 `if` → IF, rule 2 `[a-z]+` → ID, rule 3 `[a-z]+[0-9]+` → TAG. The input is `if9`. What is returned?
(A) IF, then NUM(9). (B) ID(`if`), then NUM(9). (C) TAG(`if9`). (D) A lexical error at `9`.

Solution

(C). Longest match comes first: rule 3 matches three characters, rules 1 and 2 only two, so rule 3 wins regardless of its position. Rule order decides only between rules that match the *same* longest lexeme (that is how `if` alone becomes IF and not ID). There is no NUM rule in the list anyway, so (A) and (B) invent a token. Trap: keyword priority does not beat a longer match.

Problem: A3 (2 pts): counting Thompson's construction

Using the lecture's construction (every union and every star adds two new states and four ε-edges; concatenation merges the final state of the left part with the start state of the right part), how many states and edges does the NFA for `(a|b)*` have?
(A) 6 states, 8 edges. (B) 8 states, 10 edges. (C) 8 states, 8 edges. (D) 4 states, 6 edges.

Solution

(B). Count the operators: two symbols, one union, one star, no concatenation. States $= 2 \cdot (2 + 1 + 1) = 8$. Edges: one per symbol (2), four ε-edges per union (4) and four per star (4): 10. Nobody is merged because there is no concatenation. (A) forgets the star's two states, (D) is the minimal DFA, not Thompson's NFA. Trap: the star adds two states even when it wraps a union that already has a start and an end.

Problem: A4 (2 pts): how big is the DFA?

Let L be the strings over $\{a, b\}$ whose third symbol from the end is an a, i.e. `(a|b)*a(a|b)(a|b)`. How many states does the minimal DFA for L have?
(A) 4. (B) 5. (C) 8. (D) 16.

Solution

(C). The DFA must remember the last three symbols, and every one of the $2^3 = 8$ combinations leads to a different future (two different three-letter suffixes are distinguished by an extension that pushes the difference to the third-from-last position), so no two of these 8 states can merge. The NFA has only 4 states: this family is the standard proof that subset construction can blow up to 2^n. (A) counts the NFA's states, (B) the NFA plus a dead state. Trap: an NFA with n states gives a DFA with *up to* 2^n states, and sometimes nearly that many.

Problem: A5 (2 pts): which one is regular?

Exactly one of these languages over $\{a, b\}$ is regular. Which?
(A) $\{a^n b^n : n \geq 0\}$. (B) Strings in which the substring `ab` occurs as many times as the substring `ba`. (C) $\{a^n b^m a^n : n, m \geq 0\}$. (D) $\{ww : w \in \{a, b\}^*\}$.

Solution

(B). Between two consecutive occurrences of `ab` there must be a `ba` and vice versa, so the two counts differ by at most one, and they are equal exactly when the string is empty or starts and ends with the same letter. A DFA with five states (empty; started with a, now ending in a or b; started with b, now ending in a or b) decides that. (A), (C) and (D) all need to compare two unbounded counts, which the usual distinguishing-strings argument rules out: in (A) the prefixes a^i are pairwise distinguishable by b^i. Trap: "it counts" does not mean non-regular; what matters is whether the count is bounded.

Problem: A6 (2 pts): is ε in FIRST?

$T \to F\,T'$, $T' \to *\,F\,T' \mid \varepsilon$, $F \to$ `(` E `)` $\mid$ `id`. Which is true?
(A) $\varepsilon \in \text{FIRST}(T)$ because T' is nullable. (B) $\varepsilon \notin \text{FIRST}(T)$ because F is not nullable. (C) $\varepsilon \in \text{FIRST}(T)$ because T has two symbols on its right side. (D) $\varepsilon \in \text{FIRST}(F)$ because E may

be nullable.

Solution

(B). ε enters FIRST(T) only if a *whole* right side of T can vanish. FT' starts with F, whose right sides both begin with a terminal, so F is not nullable and the computation of FIRST(FT') stops at F: FIRST(T) = FIRST(F) = {`(`, `id`}. The nullability of T' matters for FOLLOW(F), not for FIRST(T). (D) is wrong for the same reason: F's right sides start with terminals. Trap: a nullable symbol *later* in a right side never puts ε in FIRST.

Problem: A7 (2 pts): FOLLOW through a nullable neighbour

$S \to A\,B$, $A \to a \mid \varepsilon$, $B \to C\,d$, $C \to c \mid \varepsilon$. What is FOLLOW(A)?
(A) $\{c\}$. (B) $\{c, d\}$. (C) $\{d\}$. (D) $\{\$\}$.

Solution

(B). A occurs once, in $S \to AB$, so FOLLOW(A) $\supseteq$ FIRST(B)$\setminus\{\varepsilon\}$. FIRST(B) is computed from Cd: FIRST(C) $\setminus\, \varepsilon = \{c\}$, and since C is nullable the computation continues to d: FIRST(B) = $\{c, d\}$. B itself is not nullable (d is a terminal), so FOLLOW(S) = $\{\$\}$ is *not* added. (A) stops at the first symbol of B, (D) adds FOLLOW(S) as if B could vanish. Trap: FIRST of a string walks past nullable symbols; FOLLOW of the parent is added only when the whole rest is nullable.

Problem: A8 (2 pts): the dangling else

$S \to$ if E then $S \mid$ if E then S else $S \mid$ other. Which statement is true?
(A) Left factoring makes the grammar LL(1). (B) After left factoring, $M[S', \text{else}]$ still holds two productions; no rewriting of the grammar removes the ambiguity, and practical parsers resolve it by always taking the else branch. (C) The grammar is unambiguous but left-recursive. (D) The language is not context-free.

Solution

(B). Left factoring gives $S \to \text{if}\,E\,\text{then}\,S\,S' \mid \text{other}$, $S' \to \text{else}\,S \mid \varepsilon$. Now else $\in$ FIRST($S' \to$ else S) and else $\in$ FOLLOW(S') (an inner S can be followed by the outer else), so the cell $M[S', \text{else}]$ gets both productions. The conflict is inherited from a genuine ambiguity of the grammar (if E then if E then other else other has two trees), and no LL(1) grammar exists for this language with these tokens; the standard fix is to keep $S' \to \text{else}\,S$ in the cell, binding the else to the nearest if. Trap: left factoring removes a *prefix* conflict, never an ambiguity.

Problem: A9 (2 pts): an empty cell

An LL(1) parser has E on top of the stack and the next input token is `)`. The table cell $M[E,)]$ is empty, and `)` $\in$ FOLLOW(E). What does the parser do?
(A) Pops E and continues, because `)` is in FOLLOW(E). (B) Applies $E \to \varepsilon$. (C) Reports a syntax error. (D) Skips the token `)` and retries.

Solution

(C). FOLLOW sets are used once, when the table is *built*: a production $E \to \varepsilon$ would have been placed in $M[E,)]$ for every `)` $\in$ FOLLOW(E). The cell is empty, so E has no ε-production

and no production starting with `)`: nothing can be derived, error. At parse time the only rule is: nonterminal on top, look up the cell; terminal on top, match or error. (A) and (B) invent a production that the grammar does not have. (D) is error *recovery*, which only happens after the error is reported. Trap: FOLLOW helps build the table, it is not consulted while parsing.

Problem: A10 (2 pts): Chomsky type

$S \to a\,S\,b \mid a\,b$. Which statement is correct?
(A) The grammar is regular because every right side contains at most one nonterminal. (B) The grammar is context-free, and its language is not regular. (C) The grammar is context-sensitive because terminals appear on both sides of the nonterminal. (D) The grammar is context-free and its language is regular.

Solution

(B). Both productions have a single nonterminal on the left: context-free (type 2). A *regular* grammar needs the nonterminal at one end of every right side (right-linear $A \to aB$, $A \to a$); aSb has it in the middle, so (A) fails on the definition. The language is $\{a^n b^n : n \geq 1\}$, the textbook non-regular language, so (D) fails. (C) misreads "context": the type is decided by the shape of the left side, which is a single symbol. Trap: the type of the grammar and the type of the language are different questions.

Part B: One problem per technique (60 pts)

Problem: B1 (12 pts): scanning with backup

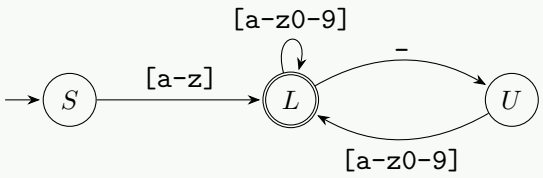
In the toy language, an *identifier* starts with a lowercase letter, continues with lowercase letters, digits and underscores, but two underscores may never be adjacent and an underscore may not be the last character. A *number* is `[0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?`. Blanks are skipped. When no rule matches at the current position, the scanner reports that one character as an error and skips it.

(a) Write a regular expression for identifiers. [3 pts]
(b) Draw a DFA for identifiers directly from the description (no Thompson, no subset construction). Say what each state remembers. [4 pts]
(c) Give the exact sequence of tokens and errors the scanner produces for the line `ab__c1 2.e5 x_`, and for every token that required *backing up*, say how many characters were read beyond the lexeme. [5 pts]

Solution

(a) `[a-z]` (`[a-z0-9]` | `_[a-z0-9]`)*. The trick is to make the underscore *come with* the character after it: then two underscores can never be adjacent, and an underscore can never be last, with one group.

(b) Three live states plus the dead state. S: nothing read. L: the last character was a letter or digit (accepting). U: the last character was an underscore (not accepting, because an identifier may not end here).



Everything not drawn goes to the dead state: a digit or _ from S, a second _ from U. The scanner backs up to the last accepting state seen, which is always an L.

(c) Longest match at each position, then rule order on ties.

Position	Output	Lexeme	Why (and backup)
`ab__c1`	ID	`ab`	$S \to L \to L \Rightarrow U \Rightarrow$dead; last accepting state was after `ab`: backs up 2 characters (`__`).
`__c1`	error	`_`	no rule starts with _; skipped.
`_c1`	error	`_`	same.
`c1`	ID	`c1`	blank ends it, no backup.
`2.e5`	NUM	`2`	`2` accepting, `2.` not (the fraction needs a digit), `e` is not allowed after `.`: dead; backs up 1 character (`.`).
`.e5`	error	`.`	no rule starts with .; skipped.
`e5`	ID	`e5`	it is a letter followed by a digit: an identifier, not an exponent.
`x_`	ID	`x`	`x` accepting, `x_` not; end of input forces the backup of 1 character.
`_`	error	`_`	skipped.

Output: ID(ab), error _, error _, ID(c1), NUM(2), error ., ID(e5), ID(x), error _. Traps: the scanner does not know that `2.e5` "was meant" as a number; it returns the longest *accepted* prefix. And end of input is just another character that does not extend the lexeme.

Problem: B2 (12 pts): subset construction and minimisation

The NFA N has states $0, 1, 2, 3$, start state 0, the single accepting state 0, and the edges

$$0 \xrightarrow{a} 2, \quad 0 \xrightarrow{b} 1, \quad 1 \xrightarrow{b} 2, \quad 1 \xrightarrow{b} 3, \quad 2 \xrightarrow{a} 0, \quad 2 \xrightarrow{a} 3, \quad 2 \xrightarrow{\varepsilon} 3.$$

(a) Give ε-closure($\{0\}$) and the DFA's start state. [2 pts]
(b) Run the subset construction. Give the full transition table, including the dead state if one is needed, and mark the accepting states. [5 pts]
(c) Minimise the DFA with the table-filling method. Show which pairs get marked in each sweep, and the resulting minimal DFA. [4 pts]
(d) Give a regular expression for $L(N)$. [1 pts]

Solution

(a) No ε-edge leaves 0, so ε-closure($\{0\}$) = $\{0\}$: the start state is $A = \{0\}$, accepting because it contains 0.

(b) For each new set and symbol: move, then closure. The only ε-edge is $2 \to 3$, so every set that contains 2 also contains 3.

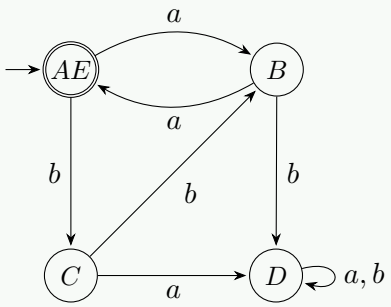
DFA state	Set	on a	on b	how
A (start, accepting)	$\{0\}$	B	C	a: $\{2\}$, closure $\{2,3\}$; b: $\{1\}$
B	$\{2,3\}$	E	D	a: $2 \to 0,3$ gives $\{0,3\}$; b: nothing leaves 2 or 3 on b: $\emptyset$
C	$\{1\}$	D	B	a: $\emptyset$; b: $\{2,3\}$, already closed
E (accepting)	$\{0,3\}$	B	C	3 contributes nothing; same rows as A
D (dead)	$\emptyset$	D	D	

Five states; A and E accepting. The dead state is a real state of the DFA: without it the table has holes and the minimisation cannot compare rows.

(c) Table filling on the 10 pairs.

- **Sweep 0** (accepting versus not): mark AB, AC, AD, EB, EC, ED. Unmarked: AE, BC, BD, CD.
- **Sweep 1**: BC: on a goes to E, D, marked $\Rightarrow$ mark BC. BD: on a goes to $E, D \Rightarrow$ mark. CD: on a both go to D; on b to B, D, *not yet* marked $\Rightarrow$ stays. AE: on a both go to B, on b both to $C \Rightarrow$ stays.
- **Sweep 2**: CD: on b to B, D, marked in sweep 1 $\Rightarrow$ mark. AE stays.
- **Sweep 3**: nothing new. Stop.

Equivalent pair: $A \equiv E$. Merge them: four states AE (start, accepting), B, C, D.



Traps: (1) the subsets $\{0\}$ and $\{0,3\}$ are *different sets* but the *same state*: NFA state 3 has no outgoing edge and is not accepting, so it never changes the future; the subset construction does not notice this, minimisation does. (2) CD is marked only in sweep 2: stopping after one sweep would leave C and the dead state merged, which is wrong (C accepts ba, D accepts nothing). You need a sweep that marks nothing.

(d) From AE: $a\,a$ returns, $b\,b\,a$ returns, anything else dies: $L(N) = (aa \mid bba)^*$. The ε-edge and the two edges into 3 were decoys.

Problem: B3 (12 pts): designing a grammar, proving and removing ambiguity

(a) Give a context-free grammar for $L = \{a^n\, b^m\, c^{n+m} : n, m \geq 0\}$ and explain in two sentences why it generates exactly L. [4 pts]

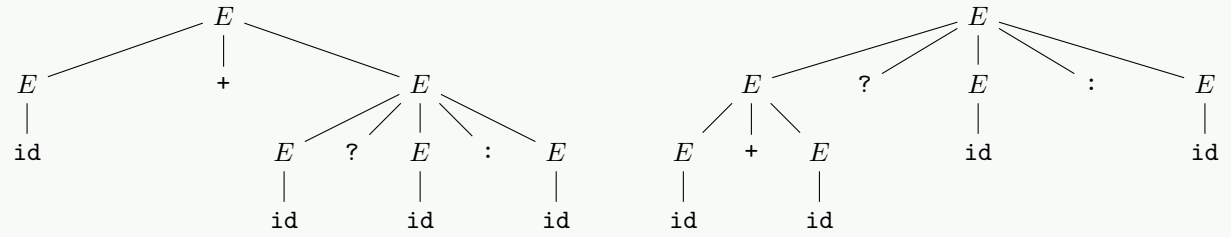
(b) Show that $E \to E$ `?` E `:` $E \mid E$ `+` $E \mid$ `id` is ambiguous, using the sentence `id + id ? id : id`. [3 pts]

(c) Write an unambiguous grammar for the same language in which `+` is left-associative and binds tighter than the conditional `? :`, and the conditional is right-associative (`a ? b : c ? d : e` means `a ? b : (c ? d : e)`). Draw the single tree of `id ? id : id ? id : id`. [5 pts]

Solution

(a) $S \to a\,S\,c \mid T$, $T \to b\,T\,c \mid \varepsilon$. Each use of $S \to aSc$ pairs one a on the left with one c on the right; after n uses, T pairs each b with a further c, inside the a-c pairs. So the string is $a^n\,b^m\,c^m\,c^n = a^n b^m c^{n+m}$, and every such string is produced by choosing n then m. Trap: $S \to A\,B$ with $A \to aAc \mid \varepsilon$ and $B \to bBc \mid \varepsilon$ generates $a^n c^n b^m c^m$, with the c's in the wrong place; the two counts must *nest*, so the b-part sits inside the a-part.

(b) Two trees for `id + id ? id : id`:

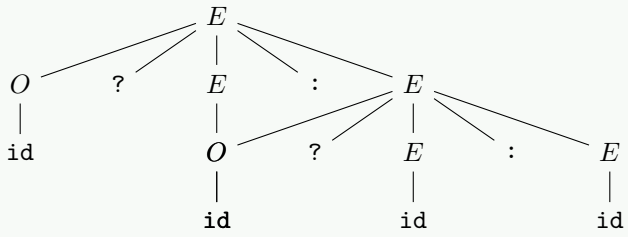


Left: `id + (id ? id : id)`; right: `(id + id) ? id : id`. Two different parse trees for one sentence: ambiguous. (A leftmost and a rightmost derivation of the same tree would prove nothing.)

(c) One nonterminal per precedence level, loosest first; the recursion side gives the associativity.

$$E \to O\ \texttt{?}\ E\ \texttt{:}\ E \mid O, \qquad O \to O\ \texttt{+}\ P \mid P, \qquad P \to \texttt{id}.$$

The atom level P holds only `id`, because the given grammar has no parentheses; writing the template's $P \to \texttt{(}\,E\,\texttt{)} \mid \texttt{id}$ here would generate strings such as `(id)` that the original cannot, so it would not be "the same language". (If parentheses were in the language, that is where they would go: $P \to \texttt{(}\,E\,\texttt{)}$, jumping back to the top level.) The condition is an O, never an E, so a conditional cannot appear bare on the left of `?`: that is what makes it right-associative, the only place a nested conditional can go is the last E. `+` is in the tighter level O, left-recursive: left-associative. Tree of `id ? id : id ? id : id` (the $O \to P \to \texttt{id}$ chains are abbreviated to `id`):



Traps: $E \to E\,\texttt{?}\,E\,\texttt{:}\,E \mid O$ keeps the ambiguity (the left E may itself be a conditional); $E \to O\,\texttt{?}\,O\,\texttt{:}\,O \mid O$ removes it but loses `a ? b : c ? d : e` altogether; and adding $P \to \texttt{(}E\texttt{)}$ by reflex changes the language. Check the terminals of the new grammar against the old one before you stop.

Problem: B4 (14 pts): the LL(1) pipeline, once

Grammar G, start symbol *Stmt*, terminals `id num = ( ) , ;`:

1	*Stmt* → `id = Exp ;`	4	*More* → `,` *Exp More* | ε
2	*Stmt* → `id ( Args ) ;`	5	*Exp* → `id` | `num`
3	*Args* → *Exp More* | ε		

(a) G is not LL(1). Name the exact table cell and the reason, and say which transformation fixes it (it is not left recursion). Write the transformed grammar G' with numbered productions. [3 pts]

(b) For G': the nullable nonterminals, then FIRST and FOLLOW of every nonterminal. [4 pts]

(c) The LL(1) table of G' (production numbers in the cells). [3 pts]

(d) The stack trace (stack, remaining input, action) for `id ( id , num ) ;`. [4 pts]

Solution

(a) $M[Stmt, \texttt{id}]$ holds both productions 1 and 2: they start with the same terminal, so one token of lookahead cannot choose. This is a *common prefix*, fixed by left factoring (left recursion would be *Stmt* → *Stmt*..., which is not the case). G':

1	*Stmt* → `id` *Rest*	5	*Args* → ε
2	*Rest* → `=` *Exp* `;`	6	*More* → `,` *Exp More*
3	*Rest* → `(` *Args* `)` `;`	7	*More* → ε
4	*Args* → *Exp More*	8, 9	*Exp* → `id`, *Exp* → `num`

(b) Nullable: *Args*, *More* (both have an ε-production; nothing else can vanish, every other right side starts with a terminal or with *Exp*).

	FIRST	FOLLOW	where FOLLOW comes from
Stmt	{`id`}	{$}	start symbol
Rest	{`=`, `(`}	{$}	last in production 1
Args	{`id`, `num`, ε}	{`)`}	followed by `)` in 3
More	{`,`, ε}	{`)`}	last in 4 and 6: inherits FOLLOW(*Args*) and FOLLOW(*More*)
Exp	{`id`, `num`}	{`;`, `,`, `)`}	three occurrences: before `;` in 2, before *More* in 4 and 6 (FIRST(*More*) = {`,`}, and *More* nullable adds FOLLOW(*Args*) = FOLLOW(*More*) = {`)`})

Trap: FOLLOW(*Exp*) needs all three occurrences; the `)` enters only through the nullable *More*.

(c) A production goes under every token of FIRST of its right side; an ε-production under every token of FOLLOW of its left side.

	`id`	`num`	`=`	`(`	`)`	`,`	`;` / $
Stmt	1						
Rest			2	3			
Args	4	4			5		
More					7	6	
Exp	8	9					

No cell holds two numbers: G' is LL(1).

(d) Stack top on the right.

Stack	Input	Action
\$ *Stmt*	`id ( id , num ) ; $`	1: *Stmt* → `id` *Rest*
\$ *Rest* `id`	`id ( id , num ) ; $`	match `id`
\$ *Rest*	`( id , num ) ; $`	3: *Rest* → `(` *Args* `) ;`
\$ `;` `)` *Args* `(`	`( id , num ) ; $`	match `(`
\$ `;` `)` *Args*	`id , num ) ; $`	4: *Args* → *Exp More*
\$ `;` `)` *More Exp*	`id , num ) ; $`	8: *Exp* → `id`
\$ `;` `)` *More* `id`	`id , num ) ; $`	match `id`
\$ `;` `)` *More*	`, num ) ; $`	6: *More* → `,` *Exp More*
\$ `;` `)` *More Exp* `,`	`, num ) ; $`	match `,`
\$ `;` `)` *More Exp*	`num ) ; $`	9: *Exp* → `num`
\$ `;` `)` *More* `num`	`num ) ; $`	match `num`
\$ `;` `)` *More*	`) ; $`	7: *More* → ε (pop only)
\$ `;` `)`	`) ; $`	match `)`
\$ `;`	`; $`	match `;`
\$	\$	accept

Traps: a right side is pushed *reversed* so that its first symbol is on top; an ε-production pops and pushes nothing; the parse accepts only when stack and input are both down to \$.

Problem: B5 (10 pts): a recursive-descent parser with two bugs

A student wrote this parser for expressions with `+`, identifiers and parentheses. `la` is the lookahead, `match(t)` consumes `t` or calls `error()`.

```
void E()  { T(); Ep(); }
void Ep() { if (la == '+')      { match('+'); T(); Ep(); }
            else if (la == ')') { match(')'); }
            else { } }
void T()  { if (la == ID)       match(ID);
            else if (la == '(') { match('('); E(); match(')'); }
            else error(); }
void parse() { la = nextToken(); E(); }
```

(a) Write the grammar the student *intended* (the usual one), with its selection sets. [2 pts]
(b) Trace the parser on the valid input `( ID )` and show where and why it fails. [3 pts]
(c) Fix `Ep()` and justify the fix with a FOLLOW set. [2 pts]
(d) With `Ep()` fixed, the parser still accepts the invalid input `ID ID`. Trace it, name the missing check, and say where the equivalent check lives in the table-driven stack parser. [3 pts]

Solution

(a) $E \to T\,E'$ (on FIRST$(T) = \{\texttt{ID}, (\}$); $E' \to \texttt{+}\,T\,E'$ (on `+`); $E' \to \varepsilon$ (on FOLLOW(E') = FOLLOW$(E) = \{), \$\}$); $T \to \texttt{ID}$ (on `ID`); $T \to \texttt{(}\,E\,\texttt{)}$ (on `(`).

(b)

```
parse()  la = '('
  E()
    T()        la = '('  -> match('('), then E()
      E()
        T()    la = ID   -> match(ID)          la becomes ')'
        Ep()   la = ')'  -> second branch: match(')')   <-- consumes the ')'
                            la becomes EOF
      match(')')  la = EOF  -> error()
```

The inner `Ep()` saw `)` and *consumed* it. But that `)` belongs to the enclosing `T()`, which was waiting to match it. When control returns to `T()`, the lookahead is EOF and `match(')')` fails: a valid input is rejected.

(c) Delete the branch `else if (la == ')') { match(')'); }`. The token `)` is in FOLLOW(E'), which is exactly the set on which $E' \to \varepsilon$ is selected: `Ep()` must return *without consuming*, and the caller consumes `)`. Rule: a token in an `if` condition is matched inside the procedure only if it begins one of that nonterminal's own right sides; a FOLLOW token is left for the caller.

(d)

```
parse()  la = ID
  E()
    T()    la = ID  -> match(ID)      la becomes ID (the second one)
    Ep()   la = ID  -> not '+': the empty branch, returns
  E() returns, parse() returns: accepted, with one ID never read.
```

The empty branch of `Ep()` accepts *any* token as ε, which is fine only if someone later checks that token. Nobody does: `parse()` must end with `if (la != EOF) error();`. In the stack parser the same check is built in: the parse accepts only when the stack top is \$ *and* the input is \$; with the input `ID ID` the stack reaches \$ while the input still holds `ID`, so the \$-row of the table reports the error. Trap: ε is selected on FOLLOW, so an ε-branch that does not test FOLLOW is a deferred check, and the start symbol's FOLLOW is \$, which only the end-of-input test can verify.

Summary: The ten traps of Part A, in one line each

A1 "bad identifier" is a parser error. A2 longest match before rule order. A3 star and union each add two states. A4 the DFA may need 2^n states. A5 a bounded count is regular. A6 $\varepsilon \in$FIRST only if a whole right side vanishes. A7 FIRST of a string walks past nullable symbols. A8 left factoring never removes an ambiguity. A9 FOLLOW builds the table, the parser only reads cells. A10 grammar type and language type are different questions.